

Applied Machine Learning Lab

B.Tech Semester 4

Experiment 11

Pneumonia Diagnosis using Machine Learning on Chest X-Ray Derived Features

Date: 22nd April 2026

SAP ID: 590016154

Name: Vidisha

AIM

To develop a machine learning-based system for pneumonia detection using numerical features extracted from chest X-ray images and evaluate different models.

THEORY

Pneumonia detection using chest X-rays can be automated using machine learning. Feature groups include global, texture (GLCM), frequency (FFT), edge and LBP features. Models used: Logistic Regression, SVM, KNN, Random Forest, Gradient Boosting. Dimensionality reduction techniques like PCA and Mutual Information are used.

Commands/Code Used

```
# Import libraries
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.feature_selection import mutual_info_classif
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.svm import SVC
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,  
confusion_matrix
```

```
# -----
```

```
# LOAD DATASET
```

```
# -----
```

```
df = pd.read_csv("train_features.csv") # update path if needed
```

```
# Target column (adjust if needed)

target_col = "label"


# -----

# FEATURE ENGINEERING

# -----

# Example: create intensity range

if "max" in df.columns and "min" in df.columns:

    df["intensity_range"] = df["max"] - df["min"]


# -----

# HANDLE MISSING VALUES

# -----

df = df.fillna(df.mean())


# -----

# SPLIT FEATURES & TARGET

# -----

X = df.drop(target_col, axis=1)

y = df[target_col]


# -----

# FEATURE SCALING

# -----

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

```

# -----

# PCA (Dimensionality Reduction)

# -----

pca = PCA(n_components=0.95)
X_pca = pca.fit_transform(X_scaled)


# Plot PCA explained variance

plt.figure(figsize=(8,5))

plt.plot(np.cumsum(pca.explained_variance_ratio_), marker='o')

plt.title("PCA Explained Variance")

plt.xlabel("Number of Components")

plt.ylabel("Cumulative Variance")

plt.grid()

plt.show()


# -----

# FEATURE SELECTION (Mutual Information)

# -----

mi_scores = mutual_info_classif(X_scaled, y)

mi_df = pd.DataFrame({"Feature": X.columns, "MI Score": mi_scores})

mi_df = mi_df.sort_values(by="MI Score", ascending=False)


# -----

# TRAIN TEST SPLIT

# -----

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X_pca, y, test_size=0.2, random_state=42
)

# -----
# MODELS
# -----

models = {
    "Logistic Regression": LogisticRegression(max_iter=500),
    "SVM": SVC(),
    "KNN": KNeighborsClassifier(),
    "Random Forest": RandomForestClassifier(),
    "Gradient Boosting": GradientBoostingClassifier()
}

results = {}

# -----
# TRAIN & EVALUATE
# -----

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)

```

```

rec = recall_score(y_test, y_pred)

f1 = f1_score(y_test, y_pred)


results[name] = [acc, prec, rec, f1]


print(f"\n{name}")
print("Accuracy:", acc)
print("Precision:", prec)
print("Recall:", rec)
print("F1 Score:", f1)


# -----
# MODEL COMPARISON
# -----

results_df = pd.DataFrame(results, index=["Accuracy", "Precision", "Recall", "F1 Score"]).T


results_df.plot(kind="bar", figsize=(10,6))
plt.title("Model Performance Comparison")
plt.xticks(rotation=45)
plt.ylabel("Score")
plt.show()


# -----
# CONFUSION MATRIX (Best Model Example)
# -----

best_model = LogisticRegression(max_iter=500)

```

```

best_model.fit(X_train, y_train)

y_pred = best_model.predict(X_test)


cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(5,4))

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")

plt.title("Confusion Matrix (Best Model)")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()


# -----

# CORRELATION HEATMAP

# -----

plt.figure(figsize=(10,8))

sns.heatmap(pd.DataFrame(X).corr(), cmap="coolwarm")

plt.title("Feature Correlation Heatmap")

plt.show()


# -----

# CLASS DISTRIBUTION

# -----

plt.figure(figsize=(6,4))

sns.countplot(x=y)

plt.title("Class Distribution")

plt.show()
```

RESULTS

Feature group analysis performed.

PCA explained variance plotted.

Class distribution analyzed.

Correlation heatmap generated.

Confusion matrix of best model obtained.

Model performance evaluated using accuracy, precision, recall and F1-score.

CONCLUSION

Machine learning models successfully detected pneumonia. Logistic Regression performed best. High recall for pneumonia class makes the model useful for medical diagnosis.